

Tokens: The Atoms of LLMs

Speaker : **Ridwan Amure**

Behind the Scenes series, Session 1

We've been teaching how to *use* LLMs and agents to build software. Now we look at how they work, starting with the smallest unit an LLM reads, writes and bills: the **token**.

Session map

#	Section	Question it answers
1	What is a token?	What does the model actually "see"?
2	Why tokenise at all?	Why not just feed it letters or words?
3	How did we get here?	The history, from file compression to GPT
4	How tokenisation works (BPE step by step)	How is a vocabulary built and used?
5	The different ways of creating tokens	BPE, WordPiece, Unigram, bytes, and beyond
6	Token quirks every builder should know	Why does the model fail at "count the r's"?
7	Tokens and the context window	What "200K context" really means
8	How token cost is calculated	What "\$2 per million tokens" means
9	Tokens in agents: where the money goes	Why agents get expensive
10	Hands-on demos	Try it yourself
11	Takeaways, discussion and glossary	

About the examples: Every tokenisation example in this doc is real output from OpenAI's open-source tokenisers (`c1100k_base` , used by GPT-4, and `o200k_base` , used by GPT-4o), run through the `tiktoken` library. Claude, Gemini and Llama each use their own tokenisers, so exact splits and counts differ between providers. The ideas are the same everywhere.

1. What is a token?

A **token** is a chunk of text that the model treats as one unit. It can be a whole word, part of a word, a single character, a space plus a word, or a piece of punctuation.

An LLM never sees letters or words. It sees a **list of integers**, where each integer is a token's ID in a fixed vocabulary.

Real examples (`c1100k_base`)

Text	Tokens (marks boundaries)	Count	Token IDs
Hello world!	Hello world !	3	9906, 1917, 0
tokenization	token ization	2	5963, 2065

Text	Tokens (marks boundaries)	Count	Token IDs
unbelievable	un belie vable	3	359, 32898, 24694
strawberry	str aw berry	3	496, 675, 15717
strawberry (with leading space)	strawberry	1	73700
Lagos	L agos	2	43, 81707
Lagos (with leading space)	Lagos	1	77096

What these show:

- **Common words are often a single token**, and the leading space is usually *part of* the token (world , not world).
- **Rarer words are split into pieces** (un + belie + vable).
- **The same word can tokenise differently depending on context.** strawberry at the start of text is 3 tokens; strawberry mid-sentence is 1.

Rules of thumb (English)

Approximation	Value
1 token	≈ 4 characters
1 token	≈ ¾ of a word
100 tokens	≈ 75 words
1,000 tokens	≈ 1.5 pages of text
1,000,000 tokens	≈ 750,000 words ≈ 8 average novels

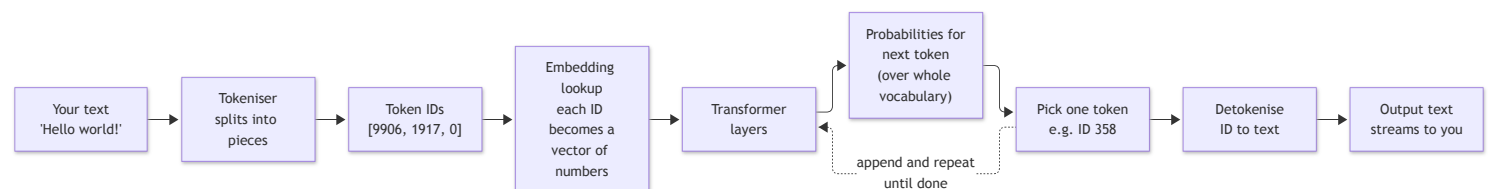
These are **English-only** estimates. Code, numbers, and most non-English languages use more tokens per word (see [Section 6](#)).

2. Why tokenise at all?

Reason 1: Neural networks only do maths on numbers

A model is a very large mathematical function. It can't take "Hello" as input. Text must become numbers, and the tokeniser does that conversion.

The full journey of your prompt



Two things to notice:

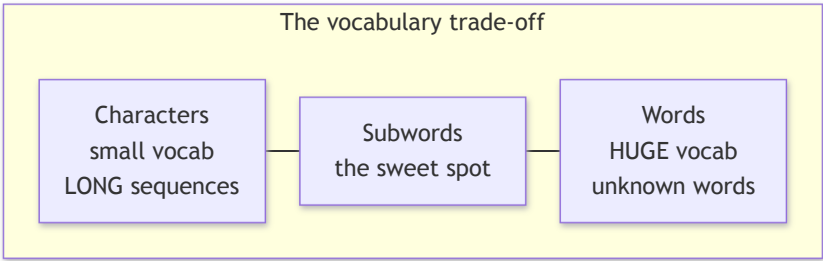
1. **The model outputs one token at a time.** Streaming responses arrive in small chunks because each chunk is a token (or a few).
2. **The loop at the bottom is why output is slow and expensive.** Each new token needs another pass through the model. More on that in [Section 8](#).

Reason 2: We need a fixed, finite vocabulary

The model's last layer produces one score for **every token in the vocabulary**. So the vocabulary must be a fixed, known list. This is the core design problem: *how do you chop up all possible text into a fixed list of pieces?*

There are three obvious options, and each has a trade-off:

Approach	Vocabulary size	"unbelievable" becomes	Strengths	Problems
Characters	Tiny (~100s)	u n b e l i e v a b l e (12)	Never meets an unknown word	Sequences get very long. The model spends effort re-learning spelling, and attention cost grows with length
Whole words	Huge (millions)	unbelievable (1), if it's in the list	Short sequences, meaningful units	New words, typos, names and slang become [UNK] (unknown). The vocabulary explodes across languages
Subwords ✓	Medium (32K–256K)	un belie vable (3)	Common words stay whole. Rare words are built from pieces. Nothing is ever unknown	Splits can look odd (see Section 6)

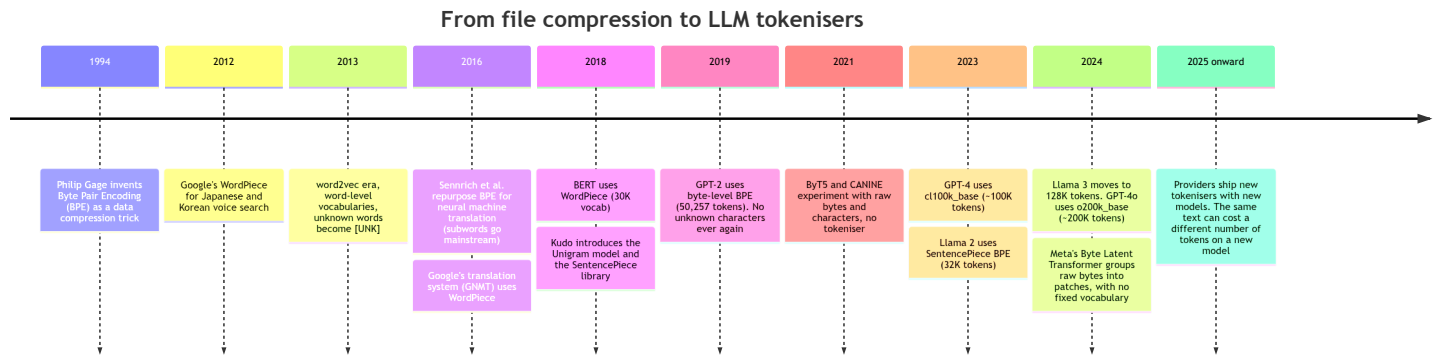


Every modern LLM uses subword tokenisation. It's the practical middle ground.

Reason 3: Compression means more meaning per step

Transformers pay for sequence length: memory and compute rise as the input gets longer. If one token carries about 4 characters instead of 1, the model can fit roughly 4× more text in the same context window, at a fraction of the compute.

3. How did we get here?



The story in plain words

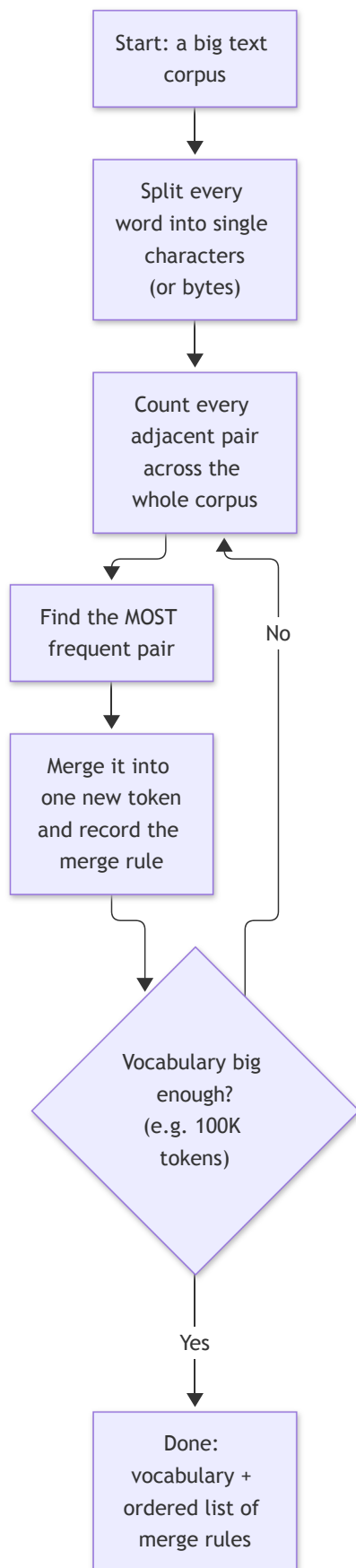
1. **Word-level era (before 2016).** Models had a fixed word list. Anything outside it (names, typos, new slang) became [UNK] , and the model lost that information.
2. **The subword breakthrough (2016).** Sennrich and colleagues took **BPE**, an old compression algorithm, and used it to build vocabularies from pieces of words. Rare words could now be spelled out from known parts, which fixed translation of names and rare words.
3. **Byte-level BPE (2019).** GPT-2 ran BPE over **raw bytes** (UTF-8) instead of characters. Any text in any script, including emoji, can now be encoded. There is no [UNK] at all.
4. **Bigger vocabularies (2023–2024).** Vocabularies grew from 32K to 50K, then 100K, 128K, 200K and 256K tokens. Bigger vocabularies mean fewer tokens per sentence, especially for code and non-English text, which makes models cheaper and faster.
5. **The frontier.** Researchers are testing models that read bytes directly and group them on the fly (for example Byte Latent Transformer). Tokenisers are still standard in production today.

4. How tokenisation works: BPE step by step

Byte Pair Encoding (BPE) is the most widely used method. It has two phases:

- **Training** (done once by the model maker): learn a list of *merge rules* from a big pile of text.
- **Encoding** (done on every request): apply those merge rules to your text.

Phase A: Training the tokeniser



Worked example (real output from a small BPE script)

Our tiny corpus has 4 words and how often each appears. `_` marks the end of a word.

Word	Frequency	Starting symbols
low	5	<code>l o w _</code>
lower	2	<code>l o w e r _</code>
newest	6	<code>n e w e s t _</code>
widest	3	<code>w i d e s t _</code>

Now we repeatedly merge the most frequent adjacent pair:

Step	Merge	Pair count	Corpus after the merge
1	<code>e + s → es</code>	9 (6 newest + 3 widest)	<code>l o w _ · l o w e r _ · n e w e s t _ · w i d e s t _</code>
2	<code>es + t → est</code>	9	<code>n e w e s t _ · w i d e s t _</code>
3	<code>est + _ → est_</code>	9	<code>n e w e s t _ · w i d e s t _</code>
4	<code>l + o → lo</code>	7	<code>l o w _ · l o w e r _</code>
5	<code>lo + w → low</code>	7	<code>l o w _ · l o w e r _</code>
6	<code>n + e → ne</code>	6	<code>n e w e s t _</code>
7	<code>ne + w → new</code>	6	<code>n e w e s t _</code>
8	<code>new + est_ → newest_</code>	6	<code>newest_</code> (a whole word is now 1 token)

Look at what the algorithm discovered on its own, without any grammar rules:

- `est_` : the English superlative suffix (*newest, widest, lowest*)
- `low` : a common root
- `newest_` : a word so frequent it earned its own token

This is exactly how real tokenisers end up with tokens like `ization`, `ing`, `the` and `function`. Real training runs over billions of words and performs 50,000 to 200,000 merges.

Phase B: Encoding new text

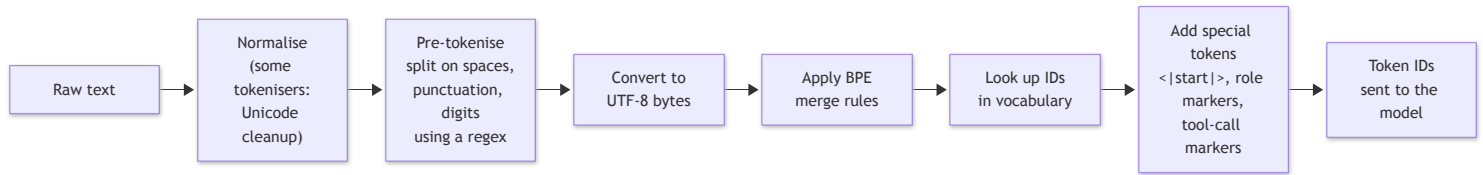
To tokenise text, apply the learned merges **in the same order**. Take the word `lowest`, which **never appeared** in our corpus:



The tokeniser handled a word it had never seen by building it from known parts. That's the main strength of subword tokenisation.

Real-world extras

Production tokenisers add a few steps around the core BPE:

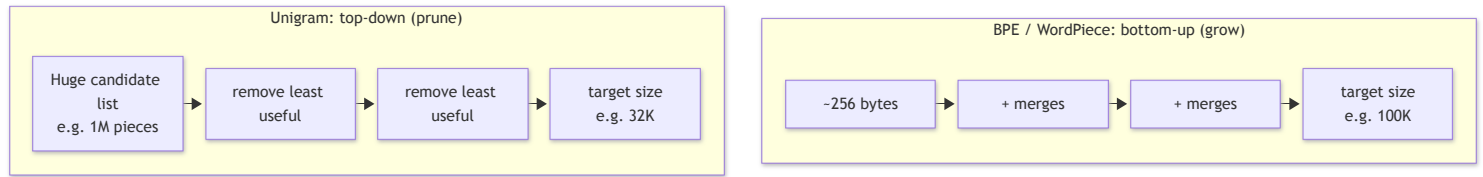


Special tokens are reserved IDs that never come from normal text. Examples are end-of-text, *"a user message starts here"* and *"a tool call starts here"*. When you send a chat request with roles (`system` , `user` , `assistant`) and tools, the provider converts it into one long token sequence with these markers. **These markers use tokens too**, which is one reason your bill is slightly higher than the visible text suggests.

5. The different ways of creating tokens

Method	How it builds the vocabulary	How it splits new text	Used by (examples)
BPE (Byte Pair Encoding)	Bottom-up: start from characters, repeatedly merge the most frequent pair	Apply merges in learned order	Original GPT, many translation models
Byte-level BPE	Same as BPE, but starts from the 256 possible bytes instead of characters	Same	GPT-2 onwards (<code>tiktoken</code>), Llama 3, most modern LLMs
WordPiece	Bottom-up, but merges the pair that most increases the likelihood of the training data (not just raw frequency). Continuation pieces are marked <code>##</code>	Greedy longest match from the left: <code>un ##believ ##able</code>	BERT, DistilBERT, ELECTRA
Unigram LM	Top-down : start with a huge candidate vocabulary and repeatedly remove the pieces that matter least	Picks the most probable split using a probabilistic model	T5, ALBERT, XLNet (via SentencePiece)
SentencePiece	A <i>library</i> , not an algorithm. Runs BPE or Unigram on raw text and treats spaces as a normal symbol (<code>_</code>), so it works for languages without spaces (Japanese, Chinese, Thai)	Depends on the algorithm chosen	Llama 1 and 2, Gemma, T5, Mistral
Character or byte-level models	No merges. Every character or byte is a token	Trivial	ByT5, CANINE (research)
Tokeniser-free or dynamic patching	The model groups raw bytes into variable-size "patches" itself, using its own predictions	Learned inside the model	Byte Latent Transformer (research)

Bottom-up vs top-down



What about Claude?

Anthropic doesn't publish Claude's tokeniser. To know exactly how many tokens something costs on Claude, use the **token counting API** (see [Section 10](#)). Tokenisers also change between model generations. Anthropic's docs note that the tokeniser introduced with Claude Opus 4.7 produces about **1.0× to 1.35× as many tokens** for the same text as earlier Claude models. **The same prompt can cost a different number of tokens after a model upgrade**, even if the price per token stays the same.

The vocabulary size trade-off

Smaller vocabulary (e.g. 32K)	Larger vocabulary (e.g. 200K)
Smaller embedding table and output layer	Bigger embedding table and output layer
More tokens per sentence, so longer sequences	Fewer tokens per sentence, so cheaper and faster per request
Each token seen more often in training	Rare tokens may be under-trained
Worse for code and non-English	Better coverage of code and many languages

6. Token quirks every builder should know

Tokenisation explains many LLM behaviours that otherwise look strange.

Quirk 1: "How many r's in strawberry?"

The model sees `str | aw | berry` (3 opaque IDs: 496, 675, 15717), not 10 letters. It never directly sees the individual `r` characters, so counting letters, reversing strings and spelling tasks are hard for it.

Builder tip: For character-level tasks (validation, counting, exact string ops), have the LLM **write and run code** instead of answering "by eye". This is one reason agents with tools do better on these tasks.

Quirk 2: Numbers are split into chunks

Text	Tokens (c1100k_base)
The cost is \$1,234,567.89	The cost is \$ 1 , 234 , 567 . 89 (11 tokens)

Numbers are chopped into groups of up to 3 digits. The model has to learn arithmetic across these arbitrary chunks, which is one reason plain LLMs make maths mistakes. **Use a calculator or code tool for exact maths.**

Quirk 3: Language inequality (the "token tax")

The same greeting, in English and in Yoruba (o200k_base):

Text	Characters	Tokens	Tokens per character
Good morning, how are you doing today?	38	9	0.24
Ẹ káàárọ̀, ẹ́é dáádáá ní lónlí?	30	19	0.63

The Yoruba sentence is **shorter** but uses **more than twice as many tokens**. The tokeniser learned mostly from English text, so accented and tonal characters are split into small pieces.

The same thing across tokeniser generations:

Text	c1100k_base (GPT-4, 2023)	o200k_base (GPT-4o, 2024)
Bawo ni? ẹ́é dáádáá ní?	16 tokens	12 tokens
How far, wetin dey happen? (Pidgin)	9 tokens (de + y)	8 tokens (dey is one token)

The newer, larger vocabulary is better for African languages, but the gap remains.

Why it matters for us:

- Apps in Yoruba, Igbo, Hausa or Pidgin **cost more per message** than the same app in English.
- They **fill the context window faster**.
- They can be **slower**, since more output tokens means more generation steps.
- Test your actual users' language when estimating cost. Don't estimate from English.

Quirk 4: Emoji and special characters can be expensive

Text	cl100k_base	o200k_base
😊🎉	5 tokens (raw byte fragments)	3 tokens

An emoji is 4 bytes in UTF-8. If the vocabulary doesn't contain it, it's split into byte-level pieces.

Quirk 5: Code and whitespace

```
def add(a, b):  
    return a + b
```

This is 11 tokens: def | add | (a | , | b |) : \n | | return | a | + | b

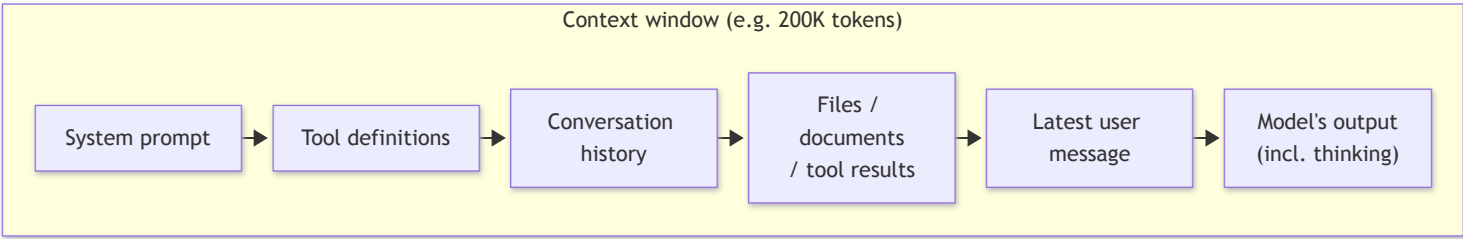
Indentation, newlines and brackets all use tokens. Minified JSON is cheaper than pretty-printed JSON. Long files full of boilerplate cost real money when an agent reads them.

Quirk 6: Leading spaces and capitalisation change tokens

Lagos = 2 tokens (L + agos), while Lagos = 1 token. Hello and hello and hello are all different IDs. To the model these are different "words" that it learned are related.

7. Tokens and the context window

The **context window** is the maximum number of tokens the model can handle in one request: **input and output combined**.



Term	Meaning
Context window	Total tokens the model can attend to at once (e.g. 200K, 1M)
Max output tokens	Cap on how many tokens the model may generate in one response (e.g. 64K, 128K)
Input tokens	Everything you send: system prompt, tools, history, files, images, the new message
Output tokens	Everything the model generates, including hidden "thinking" or reasoning tokens

Examples of how many tokens things take:

Thing	Rough token count
A short chat message	20 – 100
A typical system prompt	500 – 5,000
A tool definition (name, description, JSON schema)	100 – 1,000 each
A 300-line source file	3,000 – 5,000
A 10-page PDF of text	~6,000 – 8,000
One image	hundreds to a few thousand, depending on size

Key point: "200K context" doesn't mean 200K *words*, and it doesn't mean you should fill it. Long contexts cost more and models can get less reliable when the relevant detail is buried in a huge input.

8. How token cost is calculated

The core formula

$$\begin{aligned} \text{Cost} = & (\text{input tokens} \div 1,000,000 \times \text{input price}) \\ & + (\text{output tokens} \div 1,000,000 \times \text{output price}) \end{aligned}$$

Prices are quoted **per million tokens** (often written /MTok or /1M). Input and output have **different prices**.

What "\$2 per million tokens" means

Unit	Cost at \$2 / 1M tokens
1 token	\$0.000002
1,000 tokens	\$0.002
100,000 tokens	\$0.20
1,000,000 tokens	\$2.00

In words: **one million tokens (about 750,000 English words) of input costs \$2**. You pay for exactly the tokens you use. There's no rounding up to the next million.

Worked example 1: A single chatbot reply

Model priced at $2 / 1M \text{input} * * \text{and} * * 10 / 1M \text{output}$.

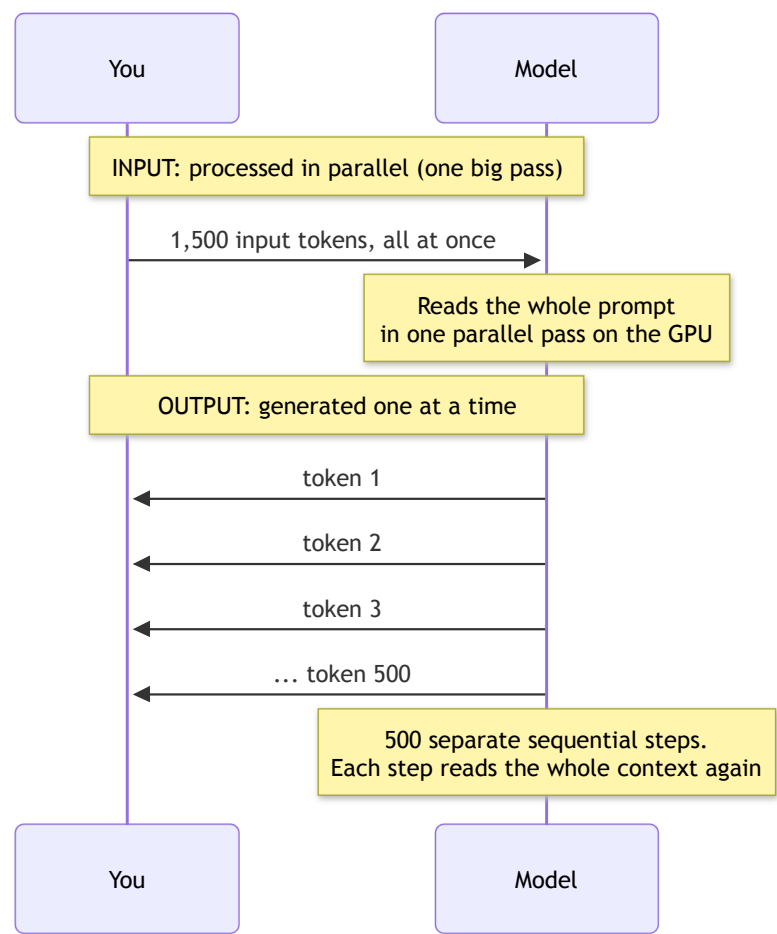
	Tokens	Calculation	Cost
Input (system prompt + history + question)	1,500	$1,500 \div 1,000,000 \times \2	\$0.003
Output (the answer)	500	$500 \div 1,000,000 \times \10	\$0.005
Total per request			\$0.008

Worked example 2: Scaling up

Requests per day	Cost per day	Cost per month (30 days)
100	\$0.80	\$24
10,000	\$80	\$2,400
1,000,000	\$8,000	\$240,000

A cost of under one cent per request is tiny for a prototype and very large at scale.

Why output tokens cost more (usually 4–5×)



- **Input** can be processed in parallel: the GPU reads all your tokens in one efficient pass.
- **Output** must be generated **one token after another**, because each new token depends on the previous one. That ties up the GPU for much longer per token.

Current real-world prices (Claude, first-party API)

Model	Input \$/1M	Output \$/1M	Output ÷ Input
Claude Haiku 4.5	\$1	\$5	5×
Claude Sonnet 5	\$2	\$10	5×
Claude Opus 5.5	\$4	\$20	5×

Model	Input \$/1M	Output \$/1M	Output ÷ Input
Claude Opus 5	\$5	\$25	5×
Claude Fable 5.1	\$10	\$50	5×

Prices from Anthropic's published rates (mid-2026). Prices change often, and every provider (OpenAI, Google, open-model hosts) publishes a similar table. Always check the provider's pricing page before quoting numbers.

Same task, different model

A task using **10,000 input + 2,000 output tokens**:

Model	Input cost	Output cost	Total
Haiku 4.5 (1/5)	\$0.010	\$0.010	\$0.020
Sonnet 5 (2/10)	\$0.020	\$0.020	\$0.040
Opus 5 (5/25)	\$0.050	\$0.050	\$0.100

Picking the right model for each job is one of the biggest cost levers you have.

Hidden token costs people forget

Hidden cost	Why it counts
System prompt	Sent (and billed) on <i>every</i> request
Tool definitions	Every tool's name, description and JSON schema is input on every call
Conversation history	Chat APIs are stateless: you resend the whole conversation each turn
Thinking / reasoning tokens	Billed as output , even when you never see them
Images and PDFs	Converted into tokens
Special / formatting tokens	Role markers and message structure add a few tokens per message
Retries and failures	A failed or retried call is still billed

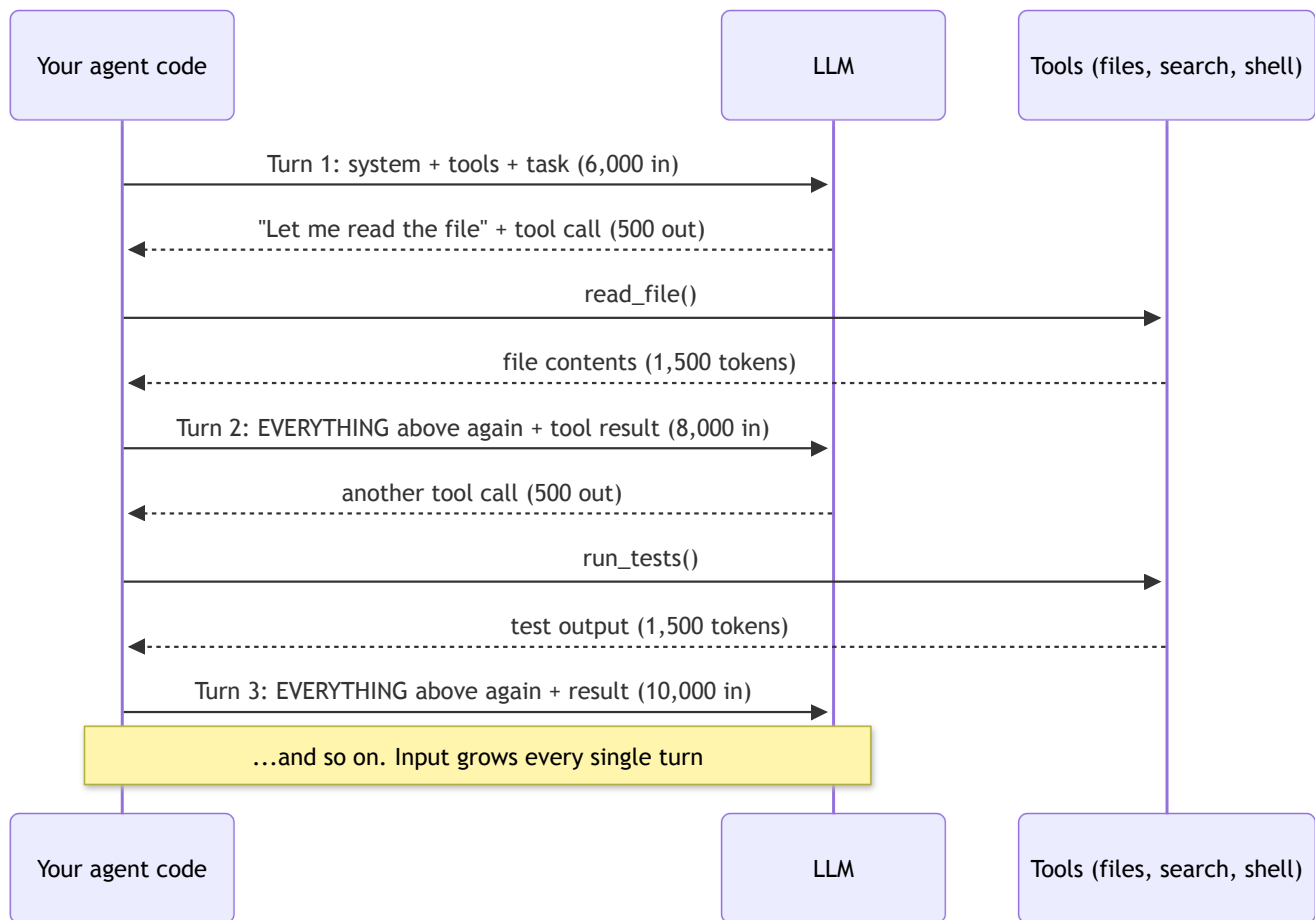
Discounts that change the maths

Feature	What it does	Typical effect
Prompt caching	The provider stores a repeated prefix (system prompt, tools, long docs). Later requests that reuse it read from cache	Cached input tokens are billed at ~10% of the normal input price (Claude). Writing to the cache costs slightly more than normal input
Batch API	Send many requests to be processed asynchronously (within 24h)	~50% off both input and output
Smaller model for simple steps	Use a cheaper model for classification, extraction or routing	2–10× cheaper for those steps
Shorter outputs	Ask for concise answers, use structured output, lower reasoning effort	Cuts the most expensive token type

9. Tokens in agents: where the money goes

This is the part that matters most for our community. **Agents are loops**, and each loop iteration **resends the entire history**.

The agent loop, in tokens



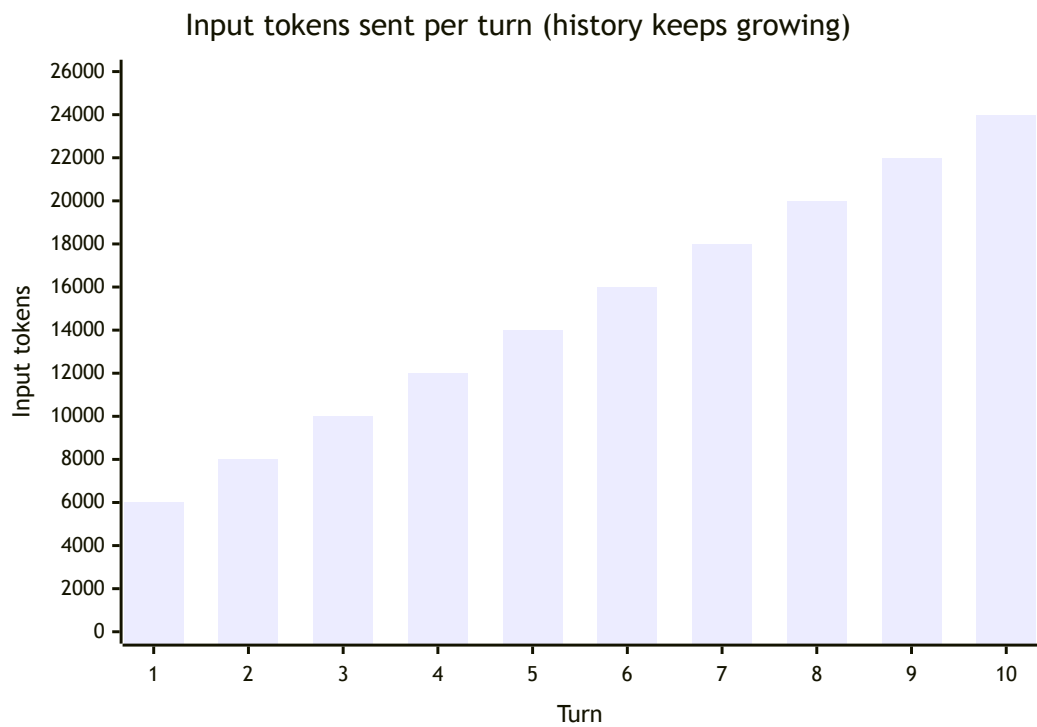
Worked example: a 10-turn coding agent

Assumptions:

- System prompt + tool definitions + task: **6,000 tokens**
- Each turn, the model outputs **500 tokens** and a tool returns **1,500 tokens**, so the history grows by **2,000 tokens per turn**
- Model: **2/1M input, 10 / 1M output**

Turn	Input tokens this turn	Output tokens
1	6,000	500
2	8,000	500
3	10,000	500
4	12,000	500
5	14,000	500
6	16,000	500

Turn	Input tokens this turn	Output tokens
7	18,000	500
8	20,000	500
9	22,000	500
10	24,000	500
Total	150,000	5,000



Cost without caching:

	Tokens	Cost
Input	150,000	$150,000 \div 1\text{M} \times 2 = \text{\textbf{**0.30**}}$
Output	5,000	$5,000 \div 1\text{M} \times 10 = \text{\textbf{**0.05**}}$
Total per task		\$0.35

Output was only 3% of the tokens, but input dominated the bill. **In agents, re-reading context is usually the main cost, not writing answers.**

Cost with prompt caching (about 126,000 of those input tokens are a repeated prefix, read from cache at ~10% price):

	Tokens	Approx. cost
New (uncached) input	~24,000	~\$0.05–0.06 (including the cache-write premium)
Cached input	~126,000	~\$0.025
Output	5,000	\$0.05
Total per task		≈ \$0.13 (about 60% cheaper)

Also note that cost grows **faster than linearly** with the number of turns. A 20-turn task at this rate sends ~500,000 input tokens, not 300,000.

Agent cost checklist

- ☐ Keep system prompts and tool lists **stable** at the start of the request so caching works
- ☐ Only give the agent the tools it needs. Every tool definition is paid for on every turn
- ☐ Trim tool outputs: return the relevant 50 lines, not the whole 5,000-line log
- ☐ Use a cheaper model for sub-tasks (search, summarise, classify)
- ☐ Use compaction or summarisation for long sessions
- ☐ Set token or turn budgets so a runaway loop can't burn money
- ☐ Log `usage` from every response so you know where tokens actually go

10. Hands-on demos

Demo 1: See tokens in the browser (no code)

- **Tiktokenizer:** <https://tiktokenizer.vercel.app>. Pick a tokeniser, paste text, and see coloured tokens with IDs.
- **OpenAI Tokenizer:** <https://platform.openai.com/tokenizer>

Try these live with the group:

1. strawberry VS strawberry
2. A sentence in English, then the same sentence in Yoruba, Igbo, Hausa or Pidgin
3. A phone number, a price, a date
4. A block of JSON: pretty-printed vs minified
5. A line of emoji

Demo 2: Tokenise with Python (tiktoken)

```
pip install tiktoken

import tiktoken

enc = tiktoken.get_encoding("o200k_base")

text = "Ṛ káàárò, ṣé dáádáá ni lóníí?"
ids = enc.encode(text)

print("Token count:", len(ids))
print("Token IDs: ", ids)
print("Pieces:     ", [enc.decode([i]) for i in ids])
print("Round trip: ", enc.decode(ids))
```

Demo 3: Count tokens for Claude

Claude's tokeniser isn't public, so use the API's token counting endpoint (free to call):

Demo 4: Read the bill from a real response

Demo 5: Build a tiny BPE yourself (about 20 lines)

```
from collections import Counter

corpus = {"low": 5, "lower": 2, "newest": 6, "widest": 3}
words = {tuple(w) + ("_",): c for w, c in corpus.items()}

for step in range(1, 9):
    pairs = Counter()
    for w, c in words.items():
        for a, b in zip(w, w[1:]):
            pairs[(a, b)] += c
    (a, b), n = pairs.most_common(1)[0]

merged = {}
for w, c in words.items():
    out, i = [], 0
    while i < len(w):
        if i < len(w) - 1 and w[i] == a and w[i + 1] == b:
            out.append(a + b); i += 2
        else:
            out.append(w[i]); i += 1
    merged[tuple(out)] = c
words = merged

print(f"step {step}: merge {a}+{b} (count {n}) ->", [" ".join(w) for w in words])
```

11. Takeaways

1. **LLMs read and write tokens, not words.** A token is a subword chunk that maps to an integer ID.
2. **Subword tokenisation is the compromise** between characters (long sequences) and words (huge vocabulary with unknown words).
3. **BPE learns its vocabulary by merging frequent pairs.** Byte-level BPE means no text is ever "unknown".
4. **Different models use different tokenisers.** The same text gives different token counts, and so different costs, across providers and even across model versions.
5. **Tokenisation explains odd behaviour:** letter counting, arithmetic slips, and the higher cost of non-English languages.
6. **Cost = tokens × price per token**, with input and output priced separately. \$2/1M means \$0.000002 per token.
7. **Output tokens cost more** because they are generated one at a time.
8. **In agents, input usually dominates the bill**, because the whole history is resent every turn. Caching, lean tools and trimmed tool outputs are your main cost levers.

Discussion questions

1. If you were building a customer-support bot for Nigerian users in Pidgin and Yoruba, how would you budget for tokens differently than for an English-only bot?
2. Why might a provider choose a 200K vocabulary over a 32K one? What do they give up?
3. Your agent's bill doubled after you added 15 new tools. Why? What would you change?
4. A model upgrade keeps the same price per token, but your monthly bill goes up 20%. What could explain it?
5. When should an agent use a code tool instead of answering directly, based on what we learned about tokens?

Glossary

Term	Meaning
Token	A chunk of text (word, subword, character or byte) treated as one unit by the model
Token ID	The integer that represents a token in the vocabulary
Vocabulary	The fixed list of all tokens a model knows (e.g. ~100K or ~200K entries)
Tokeniser	The program that converts text to token IDs and back
Encode / Decode	Text → IDs / IDs → text
BPE	Byte Pair Encoding: builds a vocabulary by repeatedly merging the most frequent adjacent pair
Byte-level BPE	BPE that starts from raw UTF-8 bytes, so any text can be encoded
WordPiece	BPE-like method that merges by likelihood gain; used by BERT
Unigram LM	Top-down method that prunes a large candidate vocabulary
SentencePiece	Library that trains BPE or Unigram directly on raw text, treating spaces as symbols
Special tokens	Reserved tokens for structure: start or end of text, roles, tool calls
Embedding	The vector of numbers the model looks up for each token ID
Context window	Max tokens (input + output) in one request
Input / output tokens	Tokens you send / tokens the model generates (billed separately)
Thinking tokens	Internal reasoning tokens a model generates before answering, billed as output
MTok	Million tokens, the unit prices are quoted in

Term	Meaning
Prompt caching	Reusing a stored prompt prefix at a large discount

Further reading

- Sennrich, Haddow & Birch (2016). *Neural Machine Translation of Rare Words with Subword Units*. The paper that brought BPE to NLP.
- Kudo & Richardson (2018). *SentencePiece: A simple and language independent subword tokenizer*.
- Radford et al. (2019). *Language Models are Unsupervised Multitask Learners* (GPT-2, byte-level BPE).
- Andrej Karpathy, *Let's build the GPT Tokenizer* (YouTube, 2024). Excellent 2-hour build-along.
- Hugging Face NLP Course, Tokenizers chapter: <https://huggingface.co/learn/nlp-course>
- Anthropic pricing: <https://www.anthropic.com/pricing>. Token counting docs: <https://docs.anthropic.com>